

HDOC – DAY 20

Application of Iterative and Decision-Making Constructs to Shapes and Physics using Modular Programming in C

Aim

To prepare an algorithm for each operation and a test-case table covering valid, invalid, and boundary inputs, then develop, compile, execute, and test a menu-driven modular C project that calculates geometrical measurements and physical properties of selected solids.

Restrictions

Use only fundamental data types, functions with call by value, selection statements, and loops. Do not use recursion, arrays, pointer variables, pointer parameters, structures, unions, enumerations, typedef, or functions from math.h. Assume that all input values and calculated results are within the ranges supported by their respective data types.

Project Organization

File	Responsibility
main.c	Displays the main menu, obtains common inputs, calls the required functions, and displays results.
geometry.c	Calculates surface area, volume, base area, and slant height.
physics.c	Obtains the solid type and required dimensions for physics operations and performs the corresponding calculations.
math_utils.c	Calculates square root iteratively using the Newton–Raphson method.
solid.h	Contains the header guard, symbolic constants, and external function declarations.

Algorithms

1. Total Surface Area and Volume of a Sphere

1. Input radius and validate $\text{radius} > 0$.
2. $\text{Area} = 4 \times \text{PI} \times \text{radius} \times \text{radius}$.
3. $\text{Volume} = (4.0 / 3.0) \times \text{PI} \times \text{radius} \times \text{radius} \times \text{radius}$.
4. Return the required result.

2. Total Surface Area and Volume of a Cylinder

1. Input radius and height; validate both are greater than 0.
2. $\text{Area} = 2 \times \text{PI} \times \text{radius} \times (\text{radius} + \text{height})$.
3. $\text{Volume} = \text{PI} \times \text{radius} \times \text{radius} \times \text{height}$.
4. Return the required result.

3. Total Surface Area and Volume of a Cone

1. Input radius and height; validate both are greater than 0.
2. Find slant height using $\text{calculate_square_root}(\text{radius} \times \text{radius} + \text{height} \times \text{height})$.
3. $\text{Area} = \text{PI} \times \text{radius} \times (\text{radius} + \text{slant height})$.
4. $\text{Volume} = (1.0 / 3.0) \times \text{PI} \times \text{radius} \times \text{radius} \times \text{height}$.
5. Return the required result.

4. Total Surface Area and Volume of a Cuboid

1. Input length, breadth and height; validate all are greater than 0.
2. $\text{Area} = 2 \times (\text{length} \times \text{breadth} + \text{breadth} \times \text{height} + \text{length} \times \text{height})$.
3. $\text{Volume} = \text{length} \times \text{breadth} \times \text{height}$.
4. Return the required result.

5. Base Area

1. For cylinder or cone, input radius and use $\text{PI} \times \text{radius} \times \text{radius}$.
2. For cuboid, input length and breadth and use $\text{length} \times \text{breadth}$.
3. A sphere is not permitted because its ideal geometrical base area is not defined by the given dimensions.

6. Mass of a Solid

1. Input density and validate $\text{density} > 0$.
2. Select the solid and obtain its dimensions.
3. Calculate volume by calling the corresponding geometry function.
4. $\text{Mass} = \text{density} \times \text{volume}$.
5. Return mass in kilograms.

7. Weight of a Solid

1. Obtain mass by calling $\text{calculate_mass}()$.
2. $\text{Weight} = \text{mass} \times \text{GRAVITY}$.
3. Return weight in newtons.

8. Gravitational Potential Energy

1. Input elevation and validate $\text{elevation} \geq 0$.
2. Obtain mass by calling $\text{calculate_mass}()$.
3. $\text{Potential energy} = \text{mass} \times \text{GRAVITY} \times \text{elevation}$.
4. Return energy in joules.

9. Pressure on the Base

1. Input density and validate $\text{density} > 0$.
2. Select cylinder, cone, or cuboid; sphere is not permitted.
3. Calculate volume and contact base area using geometry functions.
4. $\text{Pressure} = (\text{density} \times \text{volume} \times \text{GRAVITY}) / \text{contact area}$.
5. Return pressure in pascals.

10. Buoyant Force

1. Input fluid density and validate it is greater than 0.
2. Select the solid and calculate its volume.
3. Buoyant force = fluid density × volume × GRAVITY.
4. Return force in newtons.

11. Buoyancy Behaviour

1. Input solid density and fluid density; validate both are greater than 0.
2. If solid density > fluid density, return SINKS.
3. If solid density < fluid density, return FLOATS.
4. Otherwise return SUSPENDED.

12. Square Root by Newton–Raphson Method

1. If number < 0, return -1; if number = 0, return 0.
2. For a positive number, initialize old_value = number.
3. Calculate new_value = 0.5 × (old_value + number / old_value).
4. Find the absolute difference between new_value and old_value without math.h.
5. Assign new_value to old_value and repeat while difference >= TOLERANCE.
6. Return new_value.

Source Code

solid.h

```
#ifndef SOLID_H
#define SOLID_H

#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double sphere_area(double radius);
double sphere_volume(double radius);

double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);

double cone_area(double radius, double height);
double cone_volume(double radius, double height);

double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);

double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);
```

```
double calculate_mass(double density);
double calculate_weight();
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);

int determine_buoyancy(double solid_density, double fluid_density);

double calculate_square_root(double number);
```

```
#endif
```

geometry.c

```
#include "solid.h"
```

```
double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}
```

```
double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}
```

```
double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}
```

```
double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}
```

```
double cone_area(double radius, double height)
{
    double slant_height;

    slant_height = calculate_square_root(radius * radius +
                                         height * height);

    return PI * radius * (radius + slant_height);
}
```

```
double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI * radius * radius * height;
}
```

```
double cuboid_area(double length, double breadth, double height)
{
    return 2 * (length * breadth +
                breadth * height +
                length * height);
}
```

```
double cuboid_volume(double length, double breadth, double height)
{
    return length * breadth * height;
}
```

```
double circular_base_area(double radius)
{
    return PI * radius * radius;
}
```

```
double cuboid_base_area(double length, double breadth)
```

```

{
    return length * breadth;
}

```

math_utils.c

```

#include "solid.h"

double calculate_square_root(double number)
{
    double old_value, new_value, difference;

    if (number < 0)
    {
        return -1;
    }

    if (number == 0)
    {
        return 0;
    }

    old_value = number;

    do
    {
        new_value = 0.5 * (old_value + number / old_value);

        difference = new_value - old_value;

        if (difference < 0)
        {
            difference = -difference;
        }

        old_value = new_value;

    } while (difference >= TOLERANCE);

    return new_value;
}

```

physics.c

```

#include <stdio.h>
#include "solid.h"

double calculate_mass(double density)
{
    int solid;
    double radius, height, length, breadth, volume;

    do
    {
        printf("Select solid:\n");
        printf("1. Sphere\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);

        if (solid < 1 || solid > 4)
        {
            printf("Invalid solid choice.\n");
        }

    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)

```

```

{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
        if (radius <= 0)
            printf("Radius must be greater than zero.\n");
    } while (radius <= 0);
    volume = sphere_volume(radius);
}
else if (solid == CYLINDER || solid == CONE)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
        if (radius <= 0)
            printf("Radius must be greater than zero.\n");
    } while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    } while (height <= 0);

    if (solid == CYLINDER)
        volume = cylinder_volume(radius, height);
    else
        volume = cone_volume(radius, height);
}
else
{
    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);
        if (length <= 0)
            printf("Length must be greater than zero.\n");
    } while (length <= 0);

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");
    } while (breadth <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    } while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
}

return density * volume;
}

double calculate_weight()
{
    double density, mass;

    do
    {

```

```

        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);
        if (density <= 0)
            printf("Density must be greater than zero.\n");
    } while (density <= 0);

    mass = calculate_mass(density);
    return mass * GRAVITY;
}

double potential_energy(double elevation)
{
    double density, mass;

    do
    {
        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);
        if (density <= 0)
            printf("Density must be greater than zero.\n");
    } while (density <= 0);

    mass = calculate_mass(density);
    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density)
{
    int solid;
    double radius, height, length, breadth;
    double volume, contact_area;

    do
    {
        printf("Select solid:\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);

        if (solid < CYLINDER || solid > CUBOID)
            printf("Pressure cannot be calculated for this solid.\n");
    } while (solid < CYLINDER || solid > CUBOID);

    if (solid == CYLINDER || solid == CONE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
            if (radius <= 0)
                printf("Radius must be greater than zero.\n");
        } while (radius <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
            if (height <= 0)
                printf("Height must be greater than zero.\n");
        } while (height <= 0);

        contact_area = circular_base_area(radius);
        if (solid == CYLINDER)
            volume = cylinder_volume(radius, height);
        else
            volume = cone_volume(radius, height);
    }
    else

```

```

{
    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);
    } while (length <= 0);

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    contact_area = cuboid_base_area(length, breadth);
    volume = cuboid_volume(length, breadth, height);
}

return (density * volume * GRAVITY) / contact_area;
}

double buoyant_force(double fluid_density)
{
    int solid;
    double radius, height, length, breadth, volume;

    do
    {
        printf("Select solid:\n");
        printf("1. Sphere\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);

        if (solid < 1 || solid > 4)
            printf("Invalid solid choice.\n");
    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);
        volume = sphere_volume(radius);
    }
    else if (solid == CYLINDER || solid == CONE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
        } while (height <= 0);

        if (solid == CYLINDER)
            volume = cylinder_volume(radius, height);
    }
}

```

```

        else
            volume = cone_volume(radius, height);
    }
    else
    {
        do
        {
            printf("Enter length (m): ");
            scanf("%lf", &length);
        } while (length <= 0);
        do
        {
            printf("Enter breadth (m): ");
            scanf("%lf", &breadth);
        } while (breadth <= 0);
        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
        } while (height <= 0);
        volume = cuboid_volume(length, breadth, height);
    }

    return fluid_density * volume * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density)
{
    if (solid_density > fluid_density)
        return SINKS;
    else if (solid_density < fluid_density)
        return FLOATS;
    else
        return SUSPENDED;
}

```

main.c

```

#include <stdio.h>
#include "solid.h"

int main()
{
    int choice, solid, result;
    double radius, height, length, breadth;
    double density, fluid_density, solid_density, elevation;

    do
    {
        printf("\n===== SOLID GEOMETRY AND PHYSICS =====\n");
        printf("1. Surface Area\n");
        printf("2. Volume\n");
        printf("3. Base Area\n");
        printf("4. Mass\n");
        printf("5. Weight\n");
        printf("6. Gravitational Potential Energy\n");
        printf("7. Pressure on the Base\n");
        printf("8. Buoyant Force\n");
        printf("9. Buoyancy Behavior\n");
        printf("10. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                do
                {
                    printf("\nSelect solid:\n");
                    printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
                    printf("Enter choice: ");
                    scanf("%d", &solid);

```

```

        if (solid < 1 || solid > 4)
            printf("Invalid solid choice.\n");
    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
            if (radius <= 0)
                printf("Radius must be greater than zero.\n");
        } while (radius <= 0);
        printf("Surface Area = %.2lf m^2\n", sphere_area(radius));
    }
    else if (solid == CYLINDER || solid == CONE)
    {
        do { printf("Enter radius (m): "); scanf("%lf", &radius); } while (radius <= 0);
        do { printf("Enter height (m): "); scanf("%lf", &height); } while (height <= 0);
        if (solid == CYLINDER)
            printf("Surface Area = %.2lf m^2\n", cylinder_area(radius, height));
        else
            printf("Surface Area = %.2lf m^2\n", cone_area(radius, height));
    }
    else
    {
        do { printf("Enter length (m): "); scanf("%lf", &length); } while (length <= 0);
        do { printf("Enter breadth (m): "); scanf("%lf", &breadth); } while (breadth <= 0);
        do { printf("Enter height (m): "); scanf("%lf", &height); } while (height <= 0);
        printf("Surface Area = %.2lf m^2\n", cuboid_area(length, breadth, height));
    }
    break;

case 2:
    do
    {
        printf("\nSelect solid:\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);
        if (solid < 1 || solid > 4)
            printf("Invalid solid choice.\n");
    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)
    {
        do { printf("Enter radius (m): "); scanf("%lf", &radius); } while (radius <= 0);
        printf("Volume = %.2lf m^3\n", sphere_volume(radius));
    }
    else if (solid == CYLINDER || solid == CONE)
    {
        do { printf("Enter radius (m): "); scanf("%lf", &radius); } while (radius <= 0);
        do { printf("Enter height (m): "); scanf("%lf", &height); } while (height <= 0);
        if (solid == CYLINDER)
            printf("Volume = %.2lf m^3\n", cylinder_volume(radius, height));
        else
            printf("Volume = %.2lf m^3\n", cone_volume(radius, height));
    }
    else
    {
        do { printf("Enter length (m): "); scanf("%lf", &length); } while (length <= 0);
        do { printf("Enter breadth (m): "); scanf("%lf", &breadth); } while (breadth <= 0);
        do { printf("Enter height (m): "); scanf("%lf", &height); } while (height <= 0);
        printf("Volume = %.2lf m^3\n", cuboid_volume(length, breadth, height));
    }
    break;

case 3:
    do
    {
        printf("\nSelect solid:\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");

```

```

        printf("Enter choice: ");
        scanf("%d", &solid);
        if (solid < 1 || solid > 4)
            printf("Invalid solid choice.\n");
    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)
    {
        printf("Base area is not defined for a sphere.\n");
        break;
    }
    if (solid == CYLINDER || solid == CONE)
    {
        do { printf("Enter radius (m): "); scanf("%lf", &radius); } while (radius <= 0);
        printf("Base Area = %.2lf m^2\n", circular_base_area(radius));
    }
    else
    {
        do { printf("Enter length (m): "); scanf("%lf", &length); } while (length <= 0);
        do { printf("Enter breadth (m): "); scanf("%lf", &breadth); } while (breadth <= 0);
        printf("Base Area = %.2lf m^2\n", cuboid_base_area(length, breadth));
    }
    break;

case 4:
do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);
    if (density <= 0)
        printf("Density must be greater than zero.\n");
} while (density <= 0);
printf("Mass = %.2lf kg\n", calculate_mass(density));
break;

case 5:
printf("Weight = %.2lf N\n", calculate_weight());
break;

case 6:
do
{
    printf("Enter elevation (m): ");
    scanf("%lf", &elevation);
    if (elevation < 0)
        printf("Elevation cannot be negative.\n");
} while (elevation < 0);
printf("Potential Energy = %.2lf J\n", potential_energy(elevation));
break;

case 7:
do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);
    if (density <= 0)
        printf("Density must be greater than zero.\n");
} while (density <= 0);
printf("Pressure = %.2lf Pa\n", calculate_pressure(density));
break;

case 8:
do
{
    printf("Enter fluid density (kg/m^3): ");
    scanf("%lf", &fluid_density);
    if (fluid_density <= 0)
        printf("Density must be greater than zero.\n");
} while (fluid_density <= 0);
printf("Buoyant Force = %.2lf N\n", buoyant_force(fluid_density));
break;

```

```

case 9:
do
{
    printf("Enter solid density (kg/m^3): ");
    scanf("%lf", &solid_density);
    if (solid_density <= 0)
        printf("Density must be greater than zero.\n");
} while (solid_density <= 0);
do
{
    printf("Enter fluid density (kg/m^3): ");
    scanf("%lf", &fluid_density);
    if (fluid_density <= 0)
        printf("Density must be greater than zero.\n");
} while (fluid_density <= 0);

result = determine_buoyancy(solid_density, fluid_density);
if (result == SINKS)
    printf("The solid sinks.\n");
else if (result == FLOATS)
    printf("The solid floats.\n");
else
    printf("The solid remains suspended.\n");
break;

case 10:
    printf("Program ended.\n");
    break;

default:
    printf("Invalid menu choice.\n");
}

} while (choice != 10);

return 0;
}

```

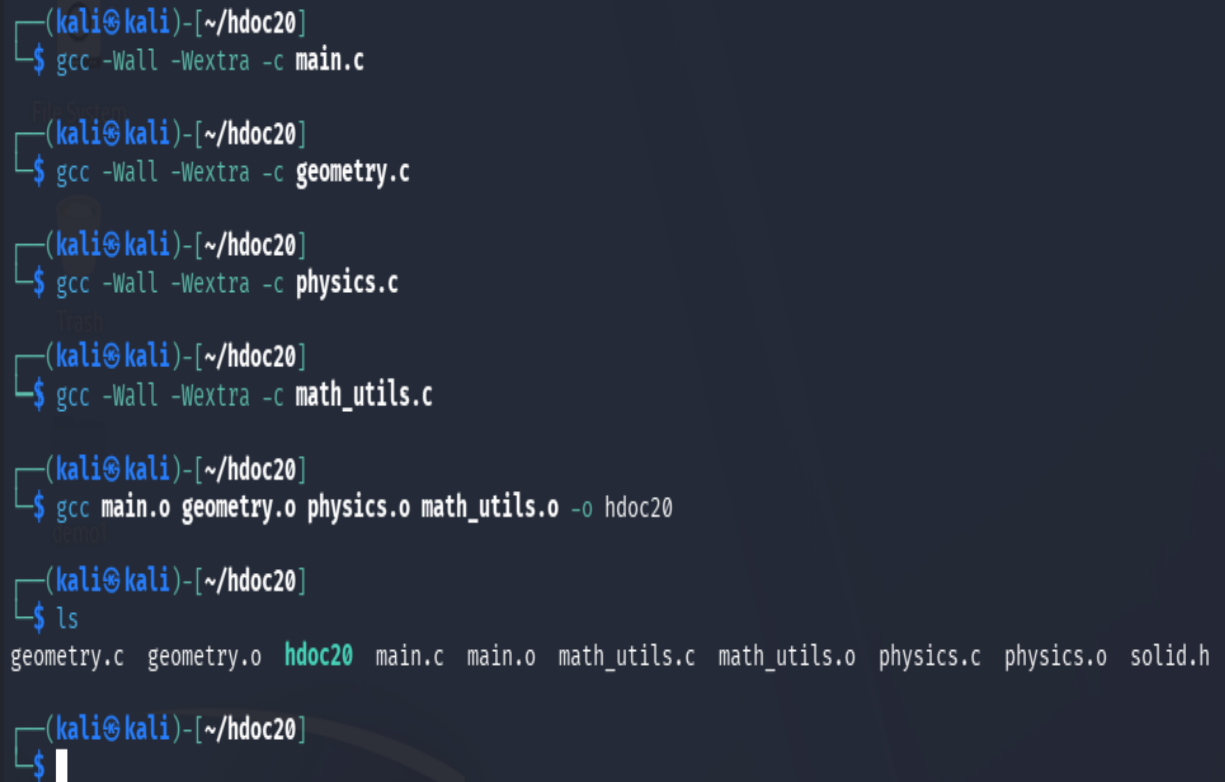
Test-Case Table

Type	Input / Solid	Choice	Expected Result
Valid	Sphere, r = 2 m	1	Surface Area = 50.27 m ²
Valid	Cylinder, r = 2 m, h = 5 m	2	Volume = 62.83 m ³
Valid	Cuboid, l = 5 m, b = 4 m	3	Base Area = 20.00 m ²
Valid	$\rho = 1000$; Sphere, r = 1 m	4	Mass = 4188.80 kg
Valid	$\rho = 1000$; Sphere, r = 1 m	5	Weight = 41050.24 N
Valid	Elevation = 10 m; $\rho = 1000$; Sphere, r = 1 m	6	Potential Energy = 410502.40 J
Valid	$\rho = 1000$; Cylinder, r = 1 m, h = 2 m	7	Pressure = 19600.00 Pa
Valid	Fluid $\rho = 1000$; Sphere, r = 1 m	8	Buoyant Force = 41050.24 N
Valid	Solid $\rho = 800$, fluid $\rho = 1000$	9	The solid floats.
Invalid	Sphere radius = 0	1	Error; asks again for radius
Invalid	Solid choice = 5	1	Invalid solid choice; asks again
Invalid	Sphere selected for base area	3	Base area is not defined for a sphere.
Boundary	Elevation = 0 m	6	Potential Energy = 0.00 J
Invalid	Any valid inputs	11	Invalid menu choice.

Compilation and Linking

The four C source modules are compiled separately with -Wall and -Wextra warning options, then linked into one executable program.

```
gcc -Wall -Wextra -c main.c
gcc -Wall -Wextra -c geometry.c
gcc -Wall -Wextra -c physics.c
gcc -Wall -Wextra -c math_utils.c
gcc main.o geometry.o physics.o math_utils.o -o hdoc20
./hdoc20
```



```
(kali㉿kali)-[~/hdoc20]
└─$ gcc -Wall -Wextra -c main.c

(kali㉿kali)-[~/hdoc20]
└─$ gcc -Wall -Wextra -c geometry.c

(kali㉿kali)-[~/hdoc20]
└─$ gcc -Wall -Wextra -c physics.c

(kali㉿kali)-[~/hdoc20]
└─$ gcc -Wall -Wextra -c math_utils.c

(kali㉿kali)-[~/hdoc20]
└─$ gcc main.o geometry.o physics.o math_utils.o -o hdoc20

(kali㉿kali)-[~/hdoc20]
└─$ ls
geometry.c  geometry.o  hdoc20  main.c  main.o  math_utils.c  math_utils.o  physics.c  physics.o  solid.h

(kali㉿kali)-[~/hdoc20]
└─$
```

Execution

1. Main Execution — All 10 Menu Choices

Choices 1–2

```
(kali㉿kali)-[~/hdoc20]
$ ./hdoc20

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 1

Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 1
Enter radius (m): 2
Surface Area = 50.27 m^2

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice:
2

Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 2
Enter radius (m): 2
Enter height (m): 5
Volume = 62.83 m^3
```

Show: 1 → Surface Area = 50.27 m²; 2 → Volume = 62.83 m³.

Choices 3–4

===== SOLID GEOMETRY AND PHYSICS =====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice: 3

Select solid:

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

Enter choice: 4

Enter length (m): 5

Enter breadth (m): 4

Base Area = 20.00 m²

===== SOLID GEOMETRY AND PHYSICS =====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice: 4

Enter density (kg/m³): 1000

Select solid:

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

Enter choice: 1

Enter radius (m): 1

Mass = 4188.80 kg

Show: 3 → Base Area = 20.00 m²; 4 → Mass = 4188.80 kg.

Choices 5–6

===== SOLID GEOMETRY AND PHYSICS =====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice: 5

Enter density (kg/m³): 1000

Select solid:

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

Enter choice: 1

Enter radius (m): 1

Weight = 41050.24 N

===== SOLID GEOMETRY AND PHYSICS =====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice: 6

Enter elevation (m): 10

Enter density (kg/m³): 1000

Select solid:

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

Enter choice: 1

Enter radius (m): 1

Potential Energy = 410502.40 J

Show: 5 → Weight = 41050.24 N; 6 → Potential Energy = 410502.40 J.

Choices 7–8

===== SOLID GEOMETRY AND PHYSICS =====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice: 7

Enter density (kg/m³): 1000

Select solid:

2. Cylinder
3. Cone
4. Cuboid

Enter choice: 2

Enter radius (m): 1

Enter height (m): 2

Pressure = 19600.00 Pa

===== SOLID GEOMETRY AND PHYSICS =====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice: 8

Enter fluid density (kg/m³): 1000

Select solid:

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

Enter choice: 1

Enter radius (m): 1

Buoyant Force = 41050.24 N

Show: 7 → Pressure = 19600.00 Pa; 8 → Buoyant Force = 41050.24 N.

Choices 9–10

```
===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 9
Enter solid density (kg/m^3): 800
Enter fluid density (kg/m^3): 1000
The solid floats.

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 10
Program ended.
```

Show: 9 → The solid floats; 10 → Program ended.

2. Test-Case Executions — Additional Cases

Invalid Radius

```
(kali㉿kali)-[~/hdoc20]
$ ./hdoc20

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 1

Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 1
Enter radius (m): 0
Radius must be greater than zero.
Enter radius (m): 2
Surface Area = 50.27 m^2

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 10
Program ended.
```

Enter radius 0 → error message; then enter radius 2 → Surface Area = 50.27 m².

Invalid Solid Choice

```
(kali㉿kali)-[~/hdoc20]
$ ./hdoc20

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 1

Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 5
Invalid solid choice.

Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 1
Enter radius (m): 2
Surface Area = 50.27 m^2

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 10
Program ended.
```

Enter solid choice 5 → Invalid solid choice; then enter a valid solid choice.

Sphere Base Area and Boundary Elevation

```
(kali㉿kali)-[~/hdoc20]
$ ./hdoc20

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 3
demo1
Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 1
Base area is not defined for a sphere.

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 6
Enter elevation (m): 0
Enter density (kg/m^3): 1000
Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 1
Enter radius (m): 1
Potential Energy = 0.00 J

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 10
Program ended.
```

Sphere base area → not defined; elevation 0 → Potential Energy = 0.00 J.

Invalid Main Menu Choice

```
(kali@kali)-[~/hdoc20]
$ ./hdoc20

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 11
Invalid menu choice.

===== SOLID GEOMETRY AND PHYSICS =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter your choice: 10
Program ended.
```

Enter menu choice 11 → Invalid menu choice; then choice 10 → Program ended.

Result

The menu-driven modular C project was developed using separate source and header files. The modules were compiled separately, linked into one executable, and tested successfully for geometrical measurements, mass, weight, gravitational potential energy, pressure, buoyant force, buoyancy behavior, invalid input, invalid menu choice, and boundary input.